PolyVote

A Multi-Tenant, Ranked-Choice Election Platform with a Tamper-Evident Audit Trail

Project Review 2

Course: BACSE202 | Database Systems
Submitted by:
Tharakesharasu Thennarasu - 25BCE2699
M Pavan Durga Karthik - 25BCE2712
Shivam Malge - 25BCE0386

Faculty: Dr. Katari Balakrishna

Abstract

Campus and organizational elections are commonly conducted using simple plurality-based voting systems that record a single preference per voter and declare the candidate with the most votes as the winner. While easy to implement, this method can produce outcomes that do not reflect majority support when three or more candidates split the vote, offers no cryptographic guarantee that recorded results have not been altered, and is typically built to serve only one election at a time. This project, PolyVote, re-engineers an earlier plurality-based voting application into a multi-tenant election management platform in which any organization can independently host elections on a shared, properly normalized relational schema. Voters rank candidates in order of preference, and results are computed using an instant-runoff (ranked-choice) tallying algorithm that better reflects overall voter preference. To guarantee integrity without violating the secret ballot, voter identity is relationally decoupled from ballot content. Every vote event is additionally sealed as a document in a cryptographically hash-chained audit ledger, making post-hoc tampering with stored ballots detectable. For this review the system is implemented with polyglot persistence on a SQLite relational database (with triggers, CHECK, UNIQUE and foreign-key constraints and analytical views), a JSON document ledger that follows the MongoDB document model, a Python FastAPI service layer, and a React web front end for voters and administrators, with the original Streamlit interface retained. Migration of the relational layer to Oracle Autonomous Database, of the ledger to MongoDB Atlas, and cloud deployment are scheduled for the final review.

1. Introduction

Elections are a foundational mechanism for democratic decision-making within academic institutions, student bodies, clubs, and similar organizations. As these processes increasingly move to digital platforms, the underlying data systems carry direct responsibility for the fairness, accuracy, and trustworthiness of outcomes. Most small-scale digital voting tools built for this purpose, however, are minimal CRUD (Create, Read, Update, Delete) applications. They rely on

a fixed set of candidates, basic application-level integer counters, and simple validation checks to guarantee correctness.

This standard architecture suffers from three critical database-design flaws:

- *Algorithmic Deficiency:* Simple plurality-based systems record a single preference per voter and declare the candidate with the most votes as the winner. This method routinely produces outcomes that do not reflect true majority support when three or more candidates split the vote.
- *Structural Rigidity:* They are typically built to serve only one election at a time. Deploying the system for a new organization requires standing up an entirely new database or hardcoding new tables.
- *Database Vulnerability:* They offer no cryptographic guarantee that recorded results have not been altered. In a standard relational database, an administrator with 'UPDATE' privileges can silently alter vote tallies, and the system provides no mechanism to detect this post-hoc tampering.

This project, PolyVote, begins from exactly such a system and asks what is required, at the database-design level, to make it trustworthy, general-purpose, and representative of true majority preference. By implementing a shared-schema multi-tenant architecture, pushing integrity checks down to the relational constraint and trigger level, and introducing a cryptographic hash chain in a document-oriented ledger, PolyVote resolves the classic election security paradox: the final tally is verifiable against sealed receipts while the individual voter's choices stay secret.

2. Literature Review

Ref	*Author(s), Year*	*Category / Theme*	*Proposed Technique*	*Dataset / Tools*	*Key Findings*	*Limitations*
[1]	Aulbach et al., 2008	Multi-Tenant Data Architecture	Schema mapping techniques for hosting multiple application tenants on a shared database.	Relational Databases (RDBMS)	Shared schema architecture significantly reduces deployment overhead for Software-as-a-Service (SaaS) platforms.	Relies entirely on standard database privileges, remaining vulnerable to internal tampering by administrators with UPDATE access.
[2]	Haber & Stornetta, 1991	Cryptographic Audit Trails	Cryptographically linking digital records via sequential hash chaining.	Digital Time-stamps, One-way Hash Functions (SHA)	Linking current records to the hash of previous records establishes a mathematically	Assumes a highly durable, trusted public repository to maintain the integrity of the

Ref	*Author(s), Year*	*Category / Theme*	*Proposed Technique*	*Dataset / Tools*	*Key Findings*	*Limitations*
					tamper-evident timeline of events.	chain over time.
[3]	Sadalage & Fowler, 2012	Polyglot Persistence	Utilizing multiple distinct database models in a single system to handle varying storage needs.	SQL and NoSQL Database Engines	Bridging strict relational constraints with document store flexibility optimizes both structured mapping and scalable logging.	Introduces higher operational complexity and requires manual programmatic management of consistency across distinct systems.
[4]	Chaum, D., 2004	Voter Privacy & Verifiability	Decoupling voter identity from ballot tracking using pseudonymous receipts.	Cryptographic Ballot Tokens	Voter verifiability can be achieved without breaking the secret ballot by separating the voter registry from the cast token.	High mathematical complexity for the end-user, often requiring specialized physical ballot printing.
[5]	Crosby & Wallach, 2009	Tamper-Evident Logging	Utilizing append-only authenticated history trees to generate cryptographic proofs of log integrity.	Merkle Trees, Cryptographic Hashes	Append-only authenticated structures allow lightweight log verification without altering existing database structures.	Validating tree structures introduces computational overhead compared to linear hash chains designed for single-sequential logs.
[6]	Blom et al., 2019	Ranked-Choice Auditing	Risk-limiting audits (RLA) and exact margin computation for Instant-Runoff Voting (IRV) algorithms.	Cast Vote Records (CVRs), Algorithmic Assorters	Statistically validating IRV elections mathematically guarantees the detection of incorrect outcomes across multiple elimination rounds.	Heavily relies on physical ballot cards for recounting and introduces steep algorithmic complexity compared to standard

Ref	*Author(s), Year*	*Category / Theme*	*Proposed Technique*	*Dataset / Tools*	*Key Findings*	*Limitations*
						plurality tallies.

3. Novelty & Research Gap (What Makes This Different)

Most academic election projects are standard CRUD applications that fail to address the complexities of real-world voting mechanisms. PolyVote introduces two major novelties, both now implemented:

1. **Algorithmic Superiority:** It replaces standard plurality "winner-takes-all" queries with a dynamic Instant-Runoff Voting (IRV) algorithm, so the elected candidate holds a majority of the continuing ballots in the final round.
2. **The Privacy/Integrity Paradox:** Standard databases cannot provide auditability without violating the secret ballot. PolyVote decouples voter identity (VOTER_PARTICIPATION) from anonymous, token-keyed ballots (BALLOT_RANKS) in the relational schema, and seals a SHA-256 digest of every ballot into a hash-chained document ledger. Any later edit or deletion of a sealed ballot in the relational database no longer matches its receipt and is reported by the integrity sweep.

4. Core Features

This project fulfills the core database management course requirements through the following implementations. The status of each is shown in brackets.

- **Normalized Multi-Tenant Relational Schema [Implemented]:** Designed up to 3NF/BCNF with entities for Organizations, Elections, Candidates, Voter Participation, Ranked Ballots and Administrator accounts. A separate POSITION entity (several posts per election) is deferred; each election currently represents one post.
- **Comprehensive Database Operations [Partially implemented]:** Full DML and TCL (transactions with COMMIT/ROLLBACK) are implemented and populated with realistic mock data (seed and bulk-generation scripts). DCL (GRANT/REVOKE) is not available in SQLite and is replaced for now by application-level roles (administrator vs. voter sessions); it will be implemented after the Oracle migration.
- **Advanced Database Logic [Partially implemented]:** A trigger strictly blocks duplicate votes, and two views provide live turnout and first-preference results. CHECK, UNIQUE, composite primary key and foreign-key constraints enforce ballot validity. Stored procedures that open and close elections automatically from timestamps are pending (lifecycle transitions are currently guarded API actions: Nominations → Voting → Closed).

- **Complex SQL Queries [Implemented]:** Multi-table joins, correlated subqueries, aggregation and views feed the dashboards, the reconciliation counts and the ranked-choice tallying engine.
- **Application Integration [Implemented]:** A React web interface talks to a FastAPI REST service, which connects to the database through Python's standard DB-API driver (sqlite3), the ODBC/JDBC-equivalent interface. The original Streamlit interface is retained and shares the same database.
- **Polyglot Persistence (NoSQL) [Partially implemented]:** The tamper-evident vote-event ledger is implemented as an append-only, hash-chained collection of JSON documents in the MongoDB document format (mongo_audit_ledger.json). Moving this collection into a MongoDB instance is scheduled for the final review.
- **Live Cloud Deployment [Pending]:** The full system runs as a local web service (one command serves both API and UI). Cloud hosting is scheduled for the final review.

5. Tech Stack

The stack below is what this review's implementation runs on. Items marked "planned" keep the Review I target for the final submission.

- **Relational Database (Primary):** SQLite 3 (SQL, triggers, views, CHECK/UNIQUE/foreign-key constraints). Planned: Oracle Cloud Autonomous Database (PL/SQL).
- **Audit Ledger (NoSQL):** Append-only JSON document collection in the MongoDB document model. Planned: MongoDB Atlas.
- **Backend / Tallying Engine:** Python 3.12 with Pandas for Instant-Runoff recalculations; FastAPI (REST API, Uvicorn server) for the service layer.
- **Frontend Interface:** React 19 + TypeScript (Vite build) for the voter booth and administrator console; the original Streamlit interface is retained.
- **Security:** Native 'hashlib' (SHA-256) for the cryptographic chain and password hashing; bearer-token sessions; server-side write lock for ledger appends.

6. High-Level Relational Schema

The relational design completely separates administrative entities from the voting mechanism to maintain privacy. Changes since Review I are noted in italics.

- ORGANIZATION: 'org_id' (PK), 'org_name' (NOT NULL), 'admin_contact'.
- ELECTION: 'election_id' (PK), 'org_id' (FK), 'status', 'start_time', 'end_time'. *Status follows the lifecycle NOMINATION → VOTING → CLOSED.*
- CANDIDATE: 'candidate_id' (PK), 'election_id' (FK), 'name' (NOT NULL), 'platform_statement'.

- VOTER_PARTICIPATION: ‘participation_id’ (PK), ‘student_reg_no’, ‘password_hash’, ‘election_id’ (FK), ‘has_voted’ (Boolean). *Stored as INTEGER with CHECK (has_voted IN (0,1)); UNIQUE (student_reg_no, election_id); trigger trg_block_duplicate_vote.*
- BALLOT_RANKS: ‘ballot_token’ (Composite PK), ‘election_id’ (FK), ‘candidate_id’ (Composite PK, FK), ‘preference_rank’ (Integer). *CHECK (preference_rank BETWEEN 1 AND 3), i.e. a top-3 ranked ballot.*
- *ADMIN_USERS (new): ‘username’ (UNIQUE), ‘password_hash’. Holds election-officer logins for the administrator console; it has no foreign keys and is never joined to ballot data.*
- *LEDGER_BLOCK (document, outside the relational schema): ‘index’, ‘timestamp’, ‘election_id’, ‘ballot_token’, ‘vote_data_hash’, ‘prev_hash’, ‘curr_hash’. One sealed receipt per ballot, logically linked by ballot_token.*

ER DIAGRAM:-

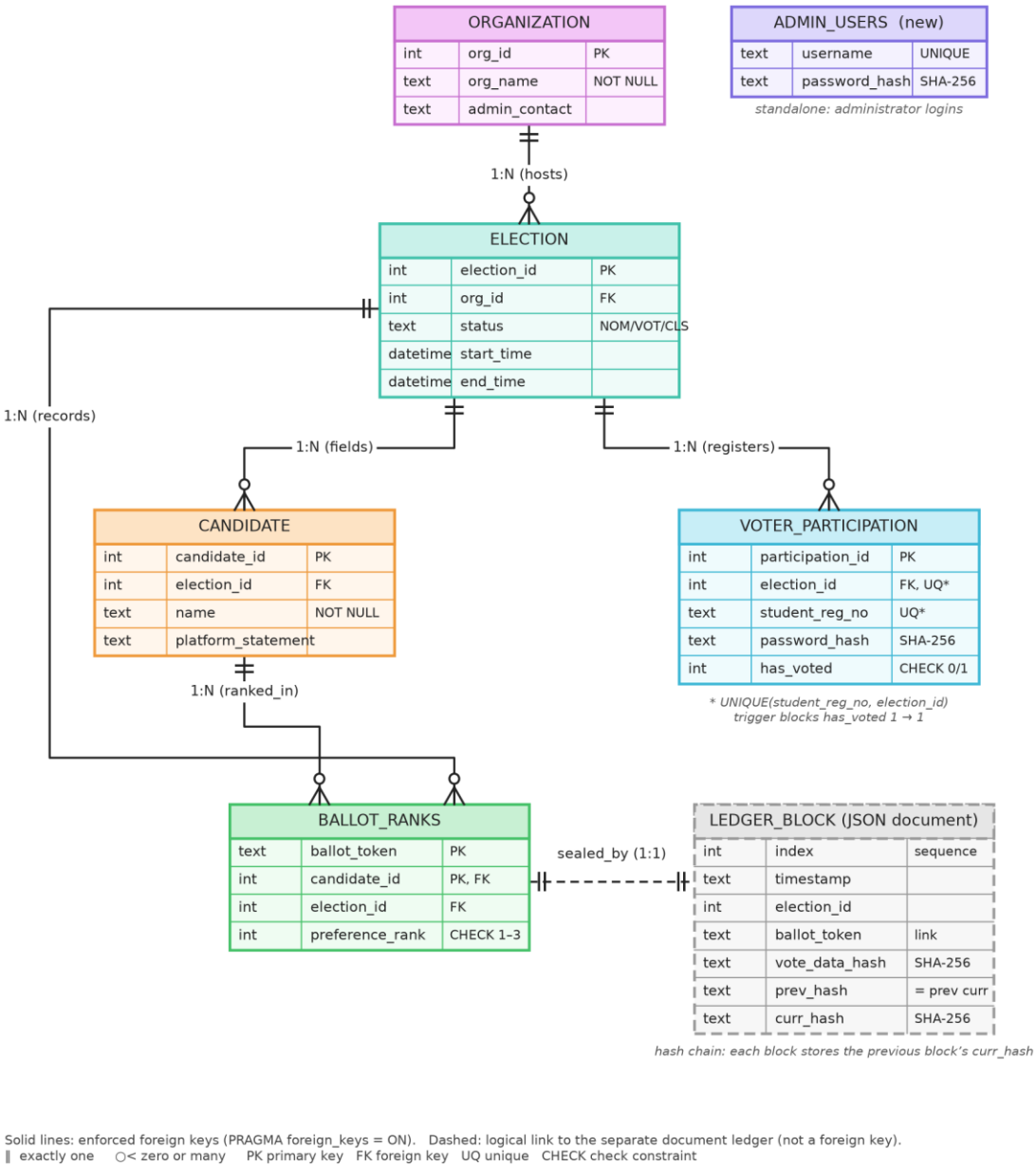


Figure 1: Updated ER diagram (ADMIN_USERS added; constraints and the document ledger shown)

7. Application Use Cases and Modifiable Capabilities

The system is designed as an "Election-as-a-Service" platform, allowing it to scale across multiple distinct environments without requiring schema redesigns or separate database deployments.

Applications:-

- Campus and Student Body Elections: Clubs, hostels, and departments can run isolated elections independently on the shared platform.

- Inter-Collegiate Federations: Affiliated chapters that require independent administrative control but share a centralized IT infrastructure.
- Hackathons and Competitions: Judging panels where ranking multiple projects is a more natural input than selecting a single winner.

Possible Modification:-

- Swappable Tallying Logic: The 'BALLOT_RANKS' schema structure is agnostic to the counting algorithm. The Instant-Runoff engine (tally_engine.py) can be swapped for Approval Voting or Borda-count scoring by altering the backend calculation function without changing the database tables.

8. System Architecture Modules

- *Module 1: Multi-Tenant Data & Relational Schema (SQLite; Oracle planned)*
    - *Function:* Acts as the primary operational database managing structured entities, organizational separation (org_id), and authorization data (voter roster and administrator accounts).
    - *Data Processing:* Before the voting session is granted, the API validates the user against the 'VOTER_PARTICIPATION' roster (registration number, password hash, has_voted) for the chosen election, which must be in the VOTING phase.
    - *Integrity Enforcement:* A unique user cannot vote more than once. UNIQUE(student_reg_no, election_id) prevents duplicate registration, and the trigger trg_block_duplicate_vote aborts any attempt to mark an already-voted participant again, without attaching the voter's identity to the cast ballot.
- *Module 2: Ranked-Choice Tallying Engine (Python/Pandas)*
    - *Function:* Executes the computational logic designed to identify a consensus winner when voters rank candidates in order of preference.
    - *Data Processing:* The engine counts each ballot's top continuing preference; if no candidate holds a majority, it eliminates the candidate with the fewest votes and redistributes those ballots to the next preferred candidate still in the race, logging every round.
    - *Edge Case Handling:* Exhausted ballots drop out of the majority threshold; elections with no ballots return an empty result; when candidates tie for last place the interface flags that the elimination was decided by engine order (a documented tie-break rule is planned).
- *Module 3: Tamper-Evident Audit Ledger (JSON documents; MongoDB planned)*
    - *Function:* Serves as the append-only, write-once, read-many (WORM) store ensuring post-election integrity.
    - *Data Processing:* Implements a continuous hash chain: each new receipt document stores the SHA-256 digest of the ballot's ranks (vote_data_hash) and

the previous block's hash (prev_hash), and is sealed with its own curr_hash. Appends are serialized by a server-side lock.

- *Verification:* audit_ledger.py re-computes every block hash and link; the integrity sweep (election_core.py) recomputes every ballot's digest from the relational tables and reports MODIFIED and DELETED ballots. Voters can check their own receipt token at /verify.

- *Module 4: Application & Deployment Layer (React + FastAPI; Streamlit retained)*
  - *Function:* The user-facing frontend and REST API orchestrate the split database read/write operations and provide the graphical interface for both administrators and voters.
  - *Data Processing:* It handles the login flow, reads the 'CANDIDATE' list, and upon submission executes a dual write: it marks the voter as voted and inserts the anonymous UUID-tagged ballot into the relational tables in one transaction, then appends the JSON receipt to the hash-chained ledger and returns the receipt token to the voter.

System Architecture Data Flow Diagram:

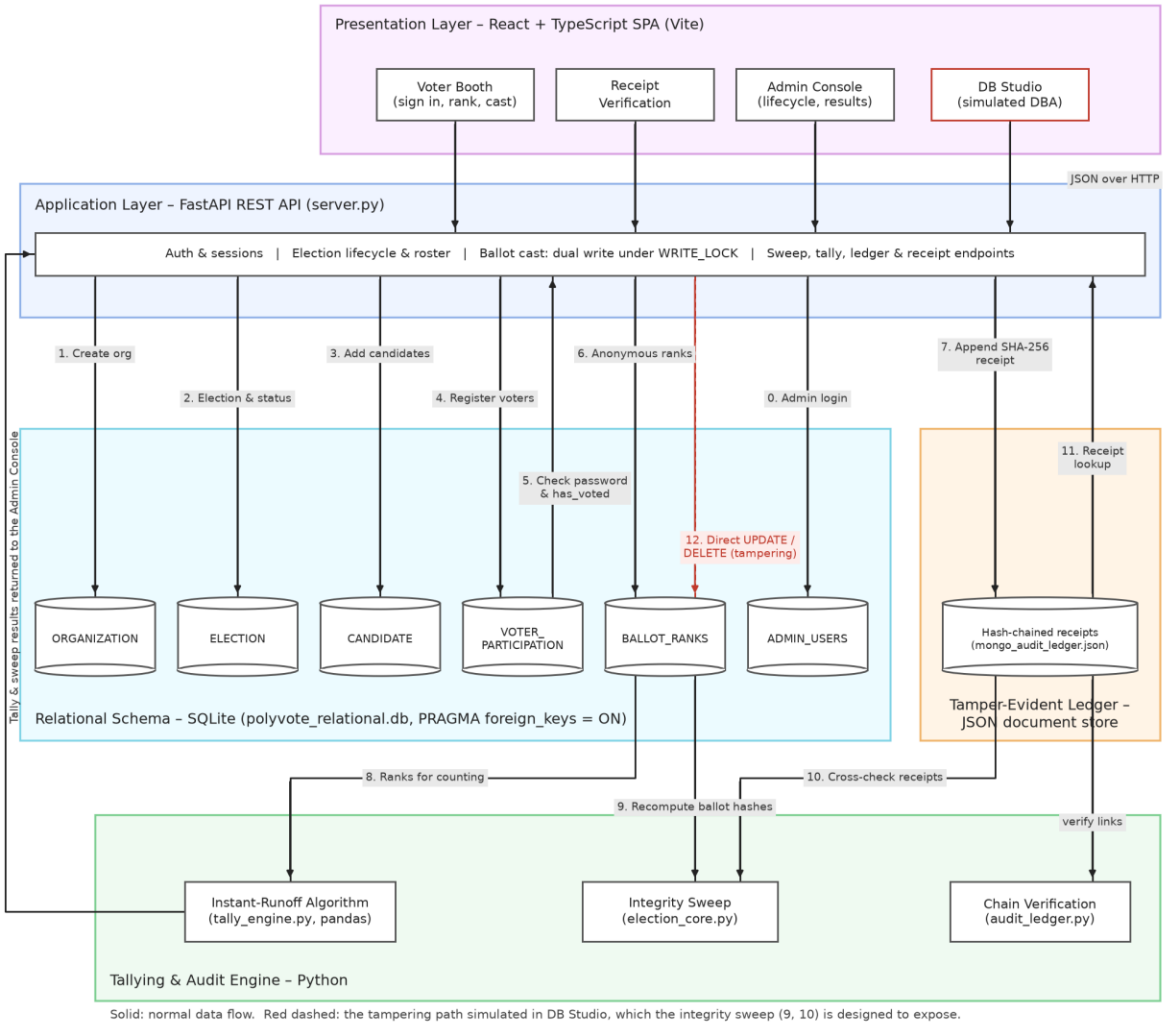


Figure 2: Updated data flow diagram (React/FastAPI layers, SQLite, JSON document ledger, tampering path)

9. Expected Final Deliverables

1. **Live Application:** A fully deployed web interface for both voters and election administrators, complete with secure authentication. *Status: complete and running locally (React + FastAPI, plus Streamlit); cloud deployment pending.*
2. **Database Scripts:** Complete DDL scripts, triggers, and mock-data generation files for initializing the relational and document stores. *Status: setup_db.py (DDL, trigger, views) and seed/bulk-generation scripts complete for SQLite; Oracle PL/SQL versions and MongoDB initialization pending.*
3. **Live Tamper-Detection & Audit Tool:** A Python verification routine that recalculates the hash chain against the live relational tables, with a simulation module that intentionally executes unauthorized UPDATE and DELETE commands to prove that anomalies are detected. *Status: complete (integrity sweep, chain verification, DB Studio tamper simulator, voter receipt check).*

10. Implementation Status

Each item promised in Review I (Sections 4, 8 and 9) was scored as implemented (1.0), partial (the share that works today) or pending (0). The overall implementation is the average of these scores.

Planned item (Review I)	Status	Score	Evidence in this review
Normalized multi-tenant schema	Implemented (POSITION entity deferred)	0.85	Section 6, Figure 1, Table 2
DML / DCL / TCL with mock data	Partial: DML, TCL done; DCL pending	0.65	Sections 11.1, 12.1
Triggers, procedures, views	Partial: trigger and 2 views done; procedures pending	0.60	Sections 11.2, 12.1
Complex SQL queries	Implemented	0.90	Section 11.3, Table 4
Application integration (DB driver)	Implemented and extended (REST API + React)	1.00	Section 12.2
NoSQL hash-chained ledger	Partial: chain and verification work; MongoDB pending	0.50	Sections 11.4, 12.1
Live cloud deployment	Pending	0.00	Section 13
Module 1: relational schema & roster	Implemented (on SQLite)	1.00	Figures 4, 9, 12
Module 2: instant-runoff engine	Implemented (tie rule pending)	0.90	Table 6, Figure 13
Module 3: audit ledger & verification	Implemented on JSON documents	0.75	Figures 15–17
Module 4: application & dual write	Implemented	1.00	Figures 5–7
Deliverable: live application	Local, not yet in the cloud	0.60	Section 12.2
Deliverable: database scripts	SQLite versions complete	0.80	Section 11
Deliverable: tamper-detection tool	Implemented	1.00	Figures 14–17

Overall: 10.55 / 14 = 75% of the Review I scope is implemented, above the 70% required for this review. All of the functional scope (multi-tenant schema, ranked ballots, instant-runoff tally, hash-chained receipts, tamper detection) works end to end. What remains is concentrated in infrastructure: Oracle PL/SQL procedures and DCL, MongoDB hosting, cloud deployment and the POSITION entity.

Beyond the Review I plan: a React voter booth and administrator console, receipt verification for voters, administrator account management, ballot simulation for load testing, a ledger explorer, reconciliation of ballot, receipt and turnout counts, and a guarded election lifecycle.

11. Implementation Details

11.1 Data definition (as created by setup_db.py)

```
CREATE TABLE BALLOT_RANKS (
        ballot_token TEXT NOT NULL,
        candidate_id INTEGER,
        election_id INTEGER,
        preference_rank INTEGER NOT NULL CHECK (preference_rank BETWEEN 1 AND 3),
        PRIMARY KEY (ballot_token, candidate_id),
        FOREIGN KEY (candidate_id) REFERENCES CANDIDATE(candidate_id),
        FOREIGN KEY (election_id) REFERENCES ELECTION(election_id)
    )

CREATE TABLE VOTER_PARTICIPATION (
        participation_id INTEGER PRIMARY KEY AUTOINCREMENT,
        election_id INTEGER,
        student_reg_no TEXT NOT NULL,
        password_hash TEXT NOT NULL,
        has_voted INTEGER DEFAULT 0 CHECK(has_voted IN (0, 1)),
        FOREIGN KEY (election_id) REFERENCES ELECTION(election_id),
        UNIQUE (student_reg_no, election_id)
    )
```

11.2 Trigger and views

```
CREATE TRIGGER trg_block_duplicate_vote
    BEFORE UPDATE OF has_voted ON VOTER_PARTICIPATION
    FOR EACH ROW
    WHEN OLD.has_voted = 1 AND NEW.has_voted = 1
    BEGIN
        SELECT RAISE(ABORT, 'DBMS Trigger Violation: Voter has already submitted a ballot.');
    END

CREATE VIEW VIEW_ELECTION_TURNOUT AS
    SELECT
        e.election_id,
        o.org_name,
        COUNT(v.participation_id) AS total_roster,
        SUM(v.has_voted) AS total_voted,
        ROUND((CAST(SUM(v.has_voted) AS FLOAT) / COUNT(v.participation_id)) * 100, 2) AS
turnout_percentage
    FROM ELECTION e
    JOIN ORGANIZATION o ON e.org_id = o.org_id
    LEFT JOIN VOTER_PARTICIPATION v ON e.election_id = v.election_id
    GROUP BY e.election_id
```

11.3 Representative query: per-election dashboard (join + correlated subqueries)

```
SELECT o.org_name, e.election_id, e.status,
  (SELECT COUNT(*) FROM CANDIDATE c WHERE c.election_id = e.election_id) AS candidates,
  (SELECT COUNT(*) FROM VOTER_PARTICIPATION v WHERE v.election_id = e.election_id) AS voters,
  (SELECT COALESCE(SUM(has_voted), 0) FROM VOTER_PARTICIPATION v
     WHERE v.election_id = e.election_id) AS voted,
  (SELECT COUNT(DISTINCT ballot_token) FROM BALLOT_RANKS b
     WHERE b.election_id = e.election_id) AS ballots
FROM ELECTION e JOIN ORGANIZATION o ON e.org_id = o.org_id;
```

11.4 Dual write and hash chaining (server.py, audit_ledger.py)

```
# one transaction in the relational store ...
conn.execute("UPDATE VOTER_PARTICIPATION SET has_voted = 1 WHERE participation_id = ?", (pid,))
conn.executemany("INSERT INTO BALLOT_RANKS (ballot_token, candidate_id, election_id,
preference_rank)"
```

```
                " VALUES (?, ?, ?, ?)", [(token, cid, e_id, rank) for cid, rank in payload])
conn.commit()
# ... then the sealed receipt in the document ledger
vote_data_hash = sha256(json.dumps(sorted(payload, key=rank)))
block = {index, timestamp, election_id, ballot_token, vote_data_hash, prev_hash: chain[-
1].curr_hash}
block["curr_hash"] = sha256(json.dumps(block, sort_keys=True))
```

11.5 Integrity sweep (election_core.py)

For every ballot token of an election, the sweep regroups the (candidate_id, preference_rank) rows from BALLOT_RANKS, recomputes their SHA-256 digest and compares it with the vote_data_hash sealed in the ledger. A mismatch is reported as MODIFIED; a receipt whose ballot no longer exists in the database is reported as DELETED. The interface also reconciles three independent counts (ballots in the database, ledger receipts, voters marked as voted), which exposes ballots that were inserted directly into the database without a receipt.

12. Outputs

12.1 Database-level outputs

Demo data: 2 organizations, 8 candidates, 60 + 30 registered voters, 45 ballots cast in VIT Student Council. The tables below were captured after the tampering demonstration of Section 12.3 (1 ballot modified, 2 deleted), so they show 43 ballots in the database against 45 voters marked as voted and 45 ledger receipts.

Table 1: Constraint and trigger enforcement. Each statement was executed against the live database and rolled back.

Rule tested	Statement	Database response
Duplicate vote (trigger)	`UPDATE VOTER_PARTICIPATION SET has_voted = 1 WHERE participation_id = 1;`	`IntegrityError: DBMS Trigger Violation: Voter has already submitted a ballot.`
Rank outside 1–3 (CHECK)	`INSERT INTO BALLOT_RANKS VALUES ('demo-token', 1, 1, 4);`	`IntegrityError: CHECK constraint failed: preference_rank BETWEEN 1 AND 3`
Same candidate twice on one ballot (composite PK)	`INSERT INTO BALLOT_RANKS VALUES ('021157cc…', 1, 1, 3);`	`IntegrityError: UNIQUE constraint failed: BALLOT_RANKS.ballot_token, BALLOT_RANKS.candidate_id`
Unknown candidate (foreign key)	`INSERT INTO BALLOT_RANKS VALUES ('demo-token', 9999, 1, 1);`	`IntegrityError: FOREIGN KEY constraint failed`

Table 2: Row counts after seeding and voting.

table_name	row_count
ORGANIZATION	2
ELECTION	2

table_name	row_count
CANDIDATE	8
VOTER_PARTICIPATION	90
BALLOT_RANKS	129
ADMIN_USERS	1

Table 3: Output of VIEW_ELECTION_TURNOUT.

election_id	org_name	total_roster	total_voted	turnout_percentage
1	VIT Student Council	60	45	75
2	Computer Science Society	30	0	0

Table 4: Output of the dashboard query in Section 11.3.

org_name	election_id	status	candidates	voters	voted	ballots
VIT Student Council	1	CLOSED	5	60	45	43
Computer Science Society	2	NOMINATION	3	30	0	0

Table 5: Anonymous ballots as stored in BALLOT_RANKS. No voter identifier is present; rows are grouped only by the random ballot token.

ballot_token	candidate_id	preference_rank
021157cc-a7b4-4984-acee-34b75a1948f3	4	1
021157cc-a7b4-4984-acee-34b75a1948f3	2	2
021157cc-a7b4-4984-acee-34b75a1948f3	1	3
050556d2-f075-4ebc-b854-63c471591ccb	3	1
050556d2-f075-4ebc-b854-63c471591ccb	1	2
050556d2-f075-4ebc-b854-63c471591ccb	4	3

Table 6: Instant-runoff rounds for VIT Student Council on the tampered data (43 ballots). Engine output: Chaitanya Reddy wins with 22 votes in Round 4!

Round	Votes per candidate (share)	Eliminated
1	Diya Sharma: 10 (23.3%) Arun Kumar: 9 (20.9%) Chaitanya Reddy: 9 (20.9%)	Bhavya Singh

Round	Votes per candidate (share)	Eliminated
	Eshaan Iyer: 8 (18.6%) Bhavya Singh: 7 (16.3%)	
2	Diya Sharma: 13 (30.2%) Arun Kumar: 11 (25.6%) Chaitanya Reddy: 10 (23.3%) Eshaan Iyer: 9 (20.9%)	Eshaan Iyer
3	Diya Sharma: 16 (37.2%) Chaitanya Reddy: 14 (32.6%) Arun Kumar: 13 (30.2%)	Arun Kumar
4	Chaitanya Reddy: 22 (51.2%) Diya Sharma: 21 (48.8%)	Majority reached

Sample ledger document (block 2):


```
{
  "index": 2,
  "timestamp": "2026-10-15T02:05:55.816572",
  "election_id": 1,
  "ballot_token": "050556d2-f075-4ebc-b854-63c471591ccb",
  "vote_data_hash": "470ef92549c7ec1d9e14aa352d7dee6b342730ae4347309a2da7573e0f32619c",
  "prev_hash": "5906eabacebd5f23d96d449be153616430ba61699be1c85647f06b3745c503ea",
  "curr_hash": "077a697b84861595dcbc5893bb65e3a41a661d371d7992cacc15660c7372fb97"
}
```


Chain verification over all 45 blocks: “Chain intact. Cryptographic ledger is valid.”

12.2 Application outputs: voter side

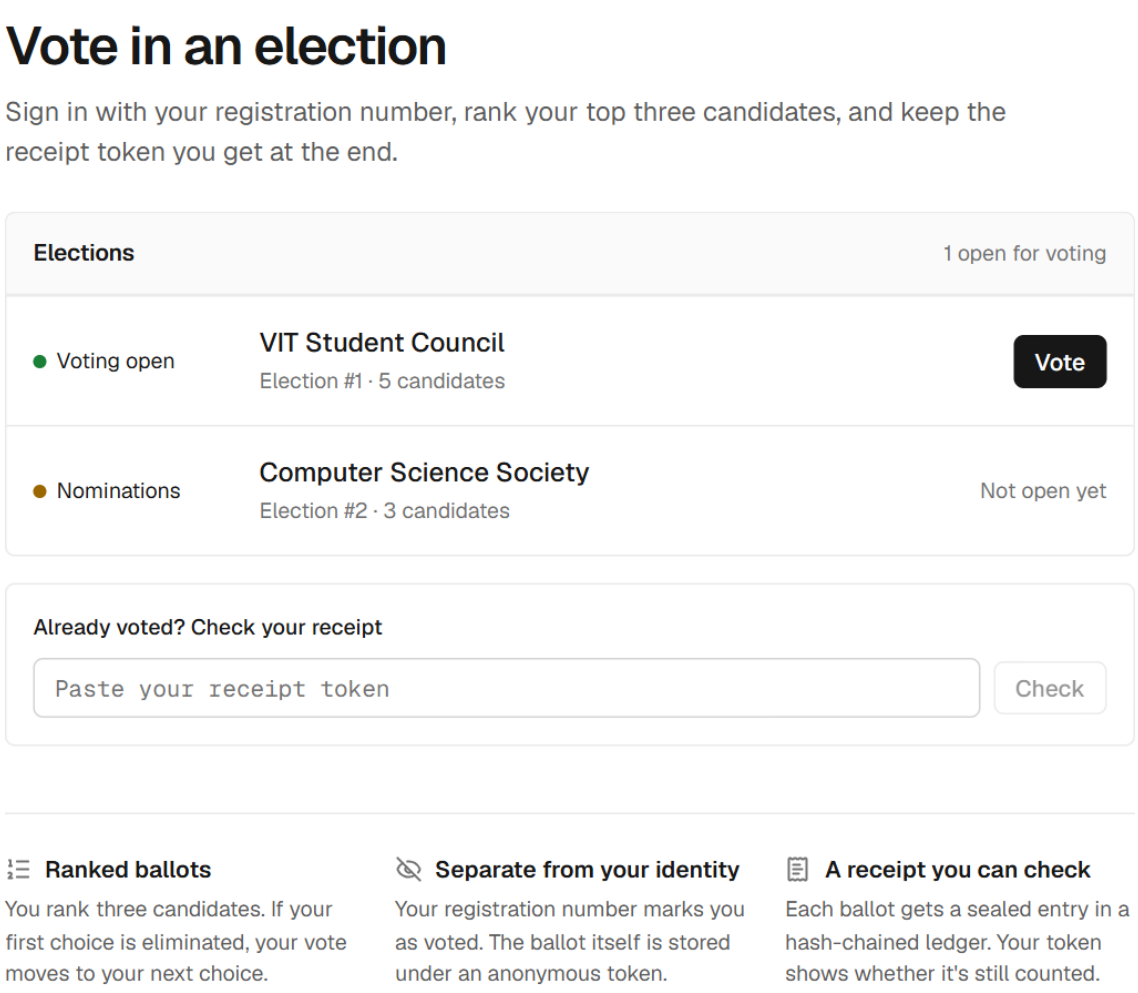


Figure 3: Voter home: open elections, inline receipt check

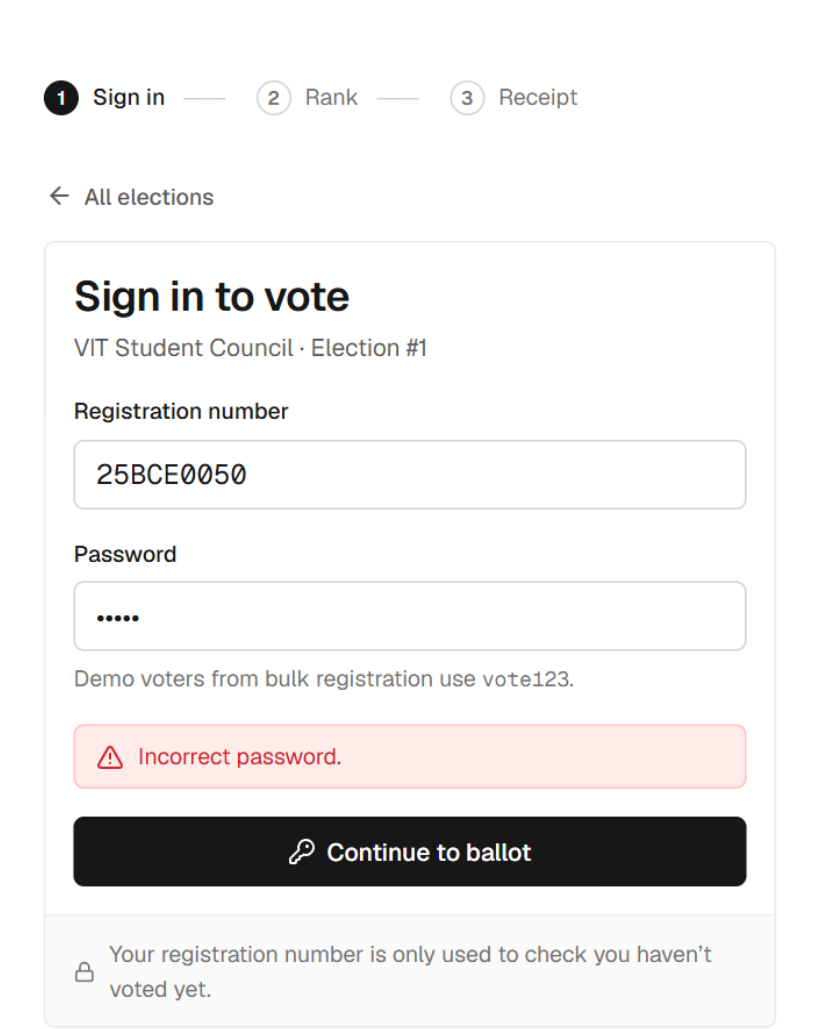


Figure 4: Voter sign-in validated against VOTER_PARTICIPATION (wrong password rejected)

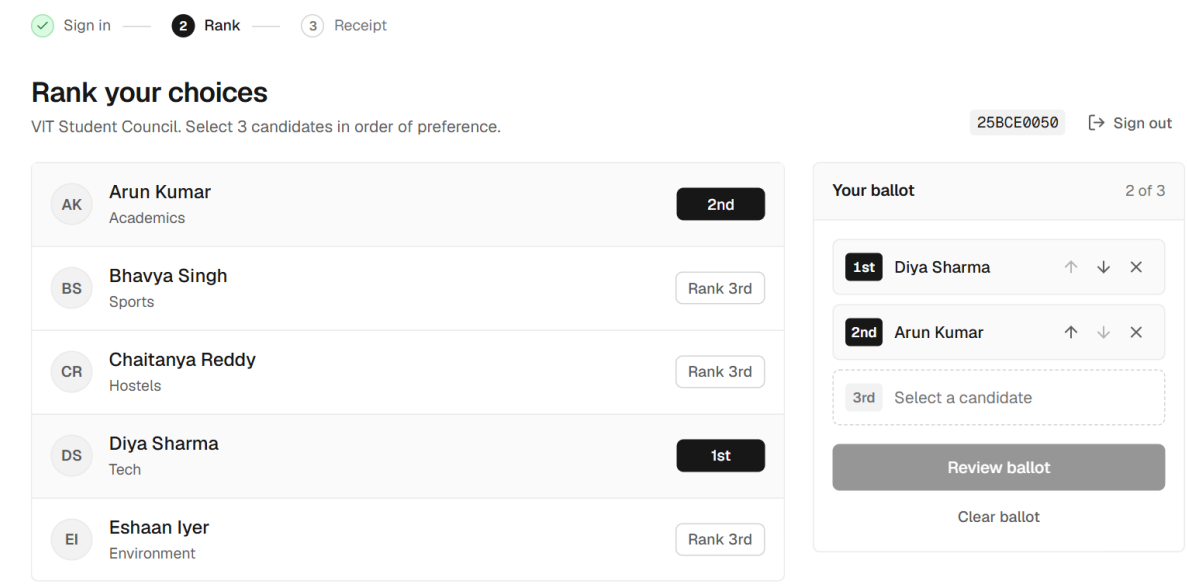


Figure 5: Ranked ballot: two of three preferences selected

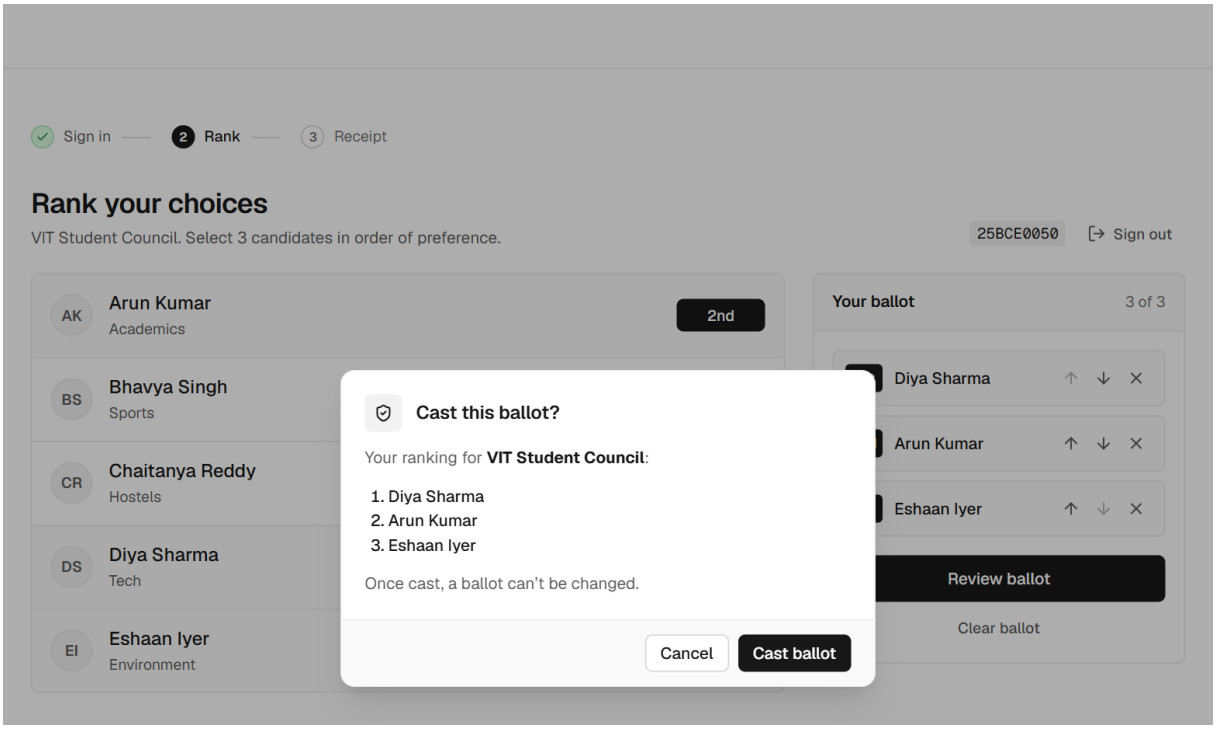


Figure 6: Confirmation before the irreversible cast

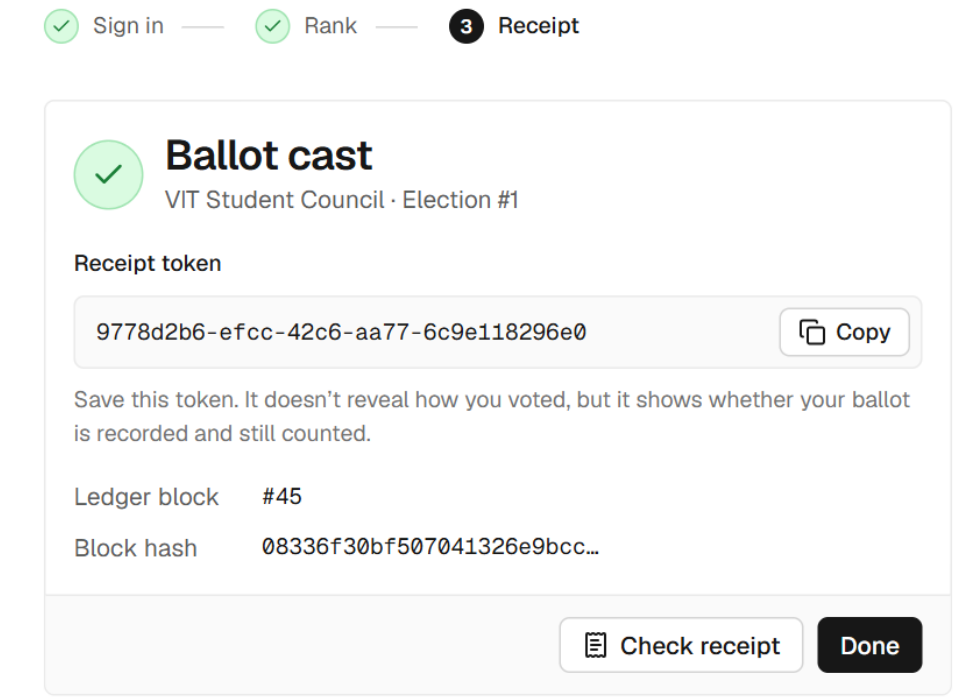


Figure 7: Receipt token and ledger block returned after the dual write

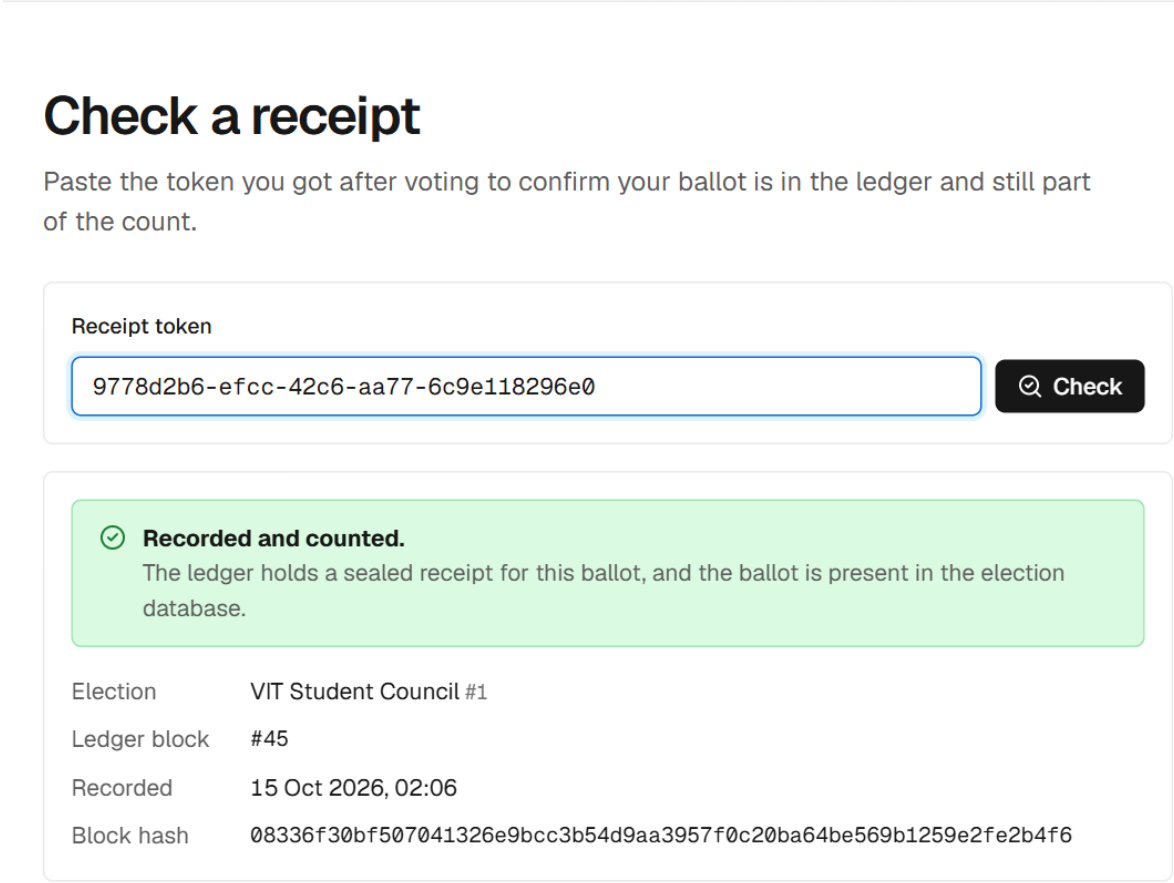


Figure 8: Receipt check: sealed in the ledger and present in the count

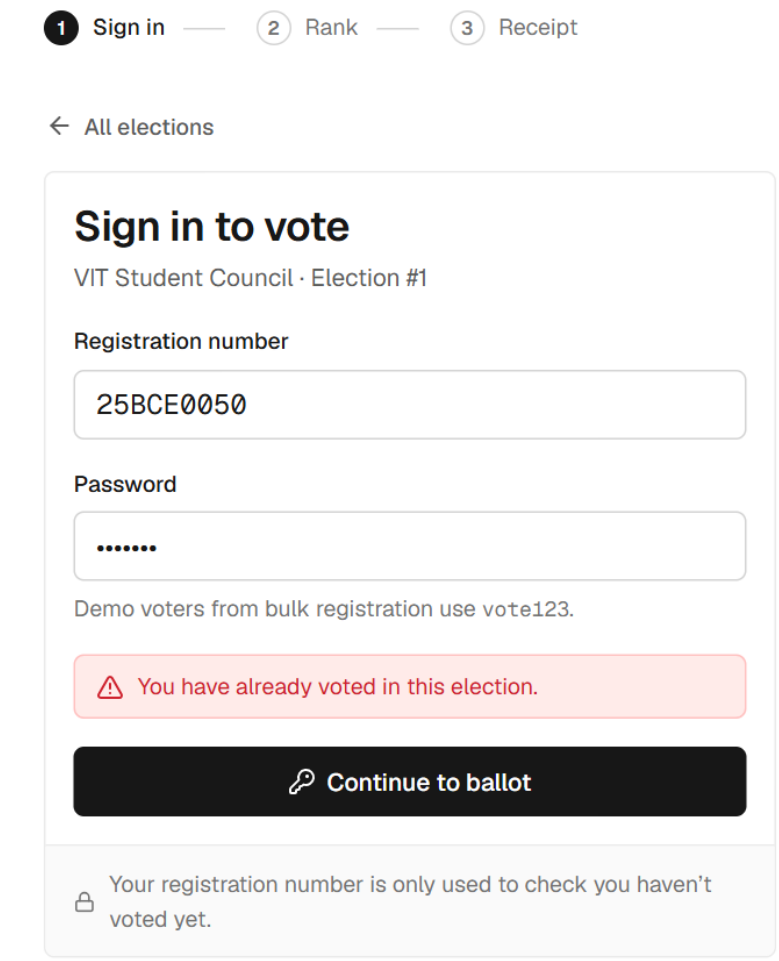


Figure 9: Second attempt by the same voter is refused

12.3 Application outputs: administrator side

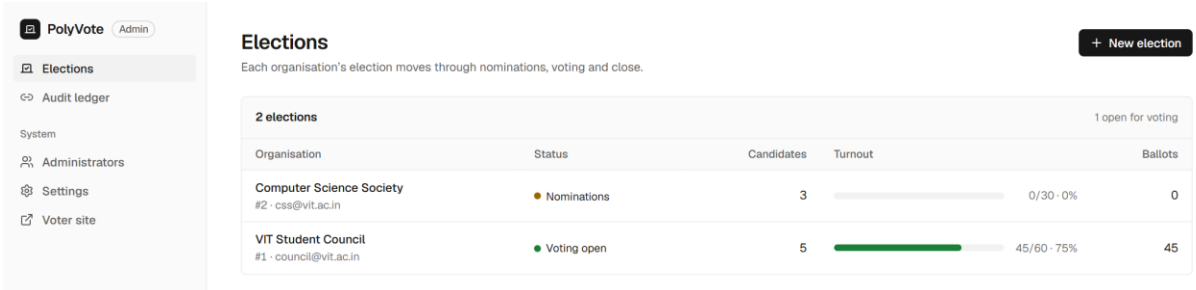


Figure 10: Multi-tenant elections list with status and turnout

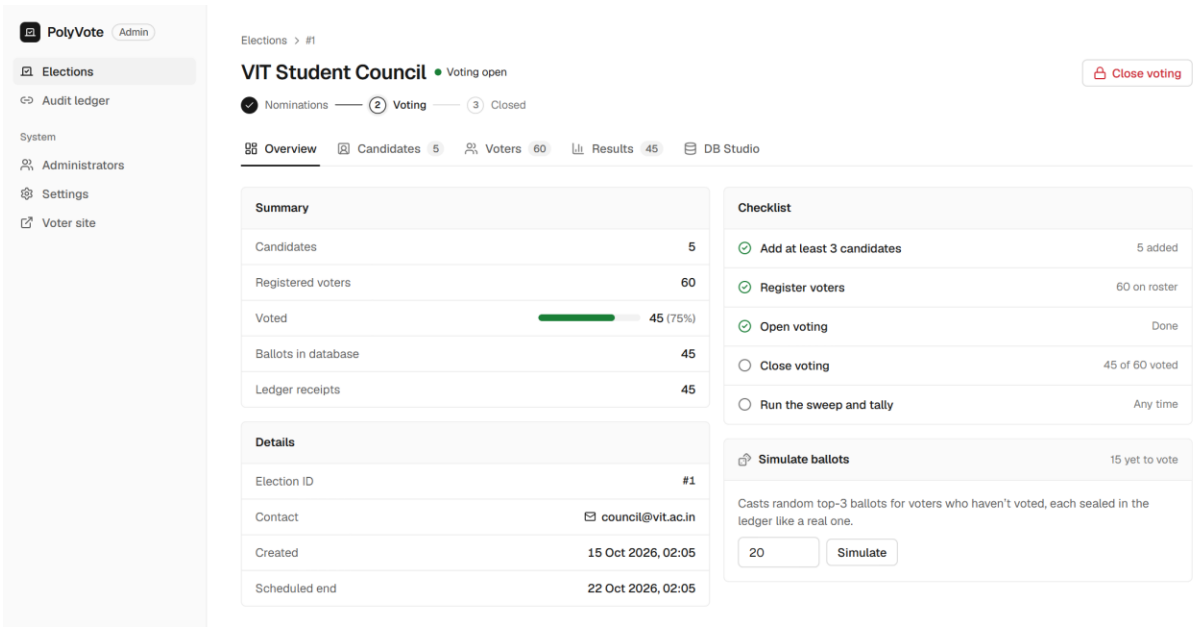


Figure 11: Election overview: summary, checklist and ballot simulation

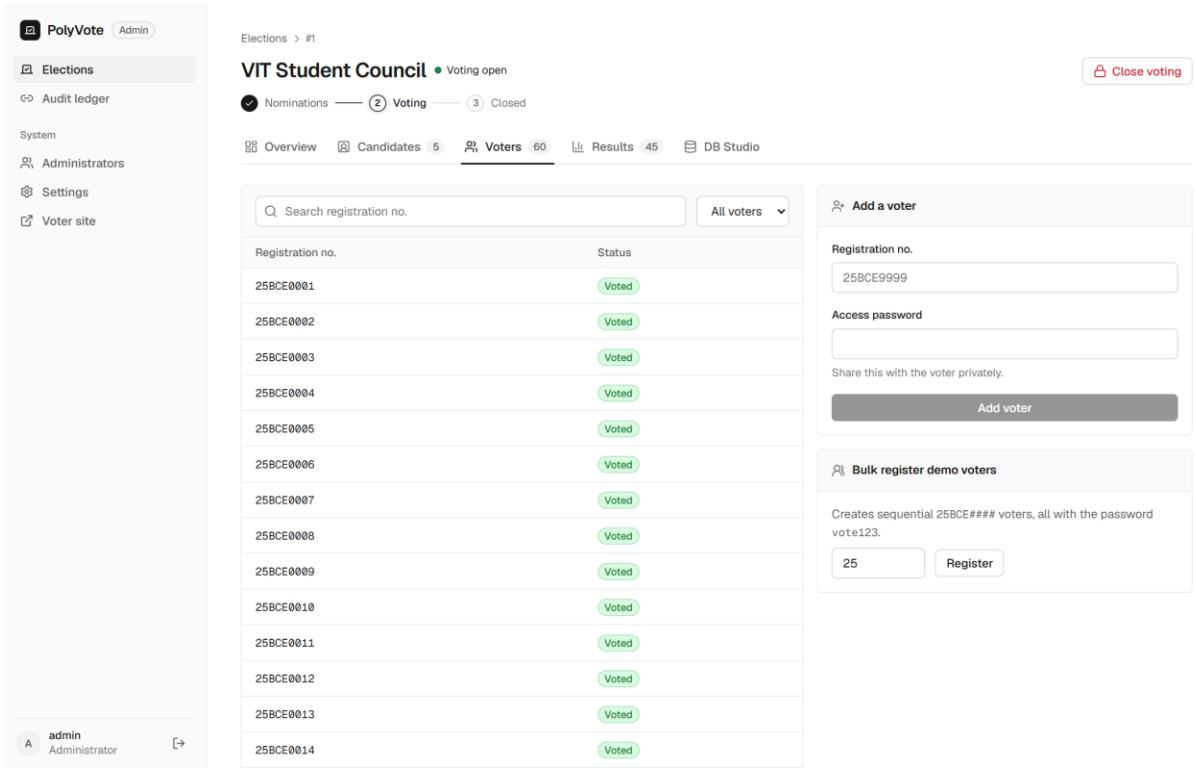


Figure 12: Voter roster with voted / not voted state

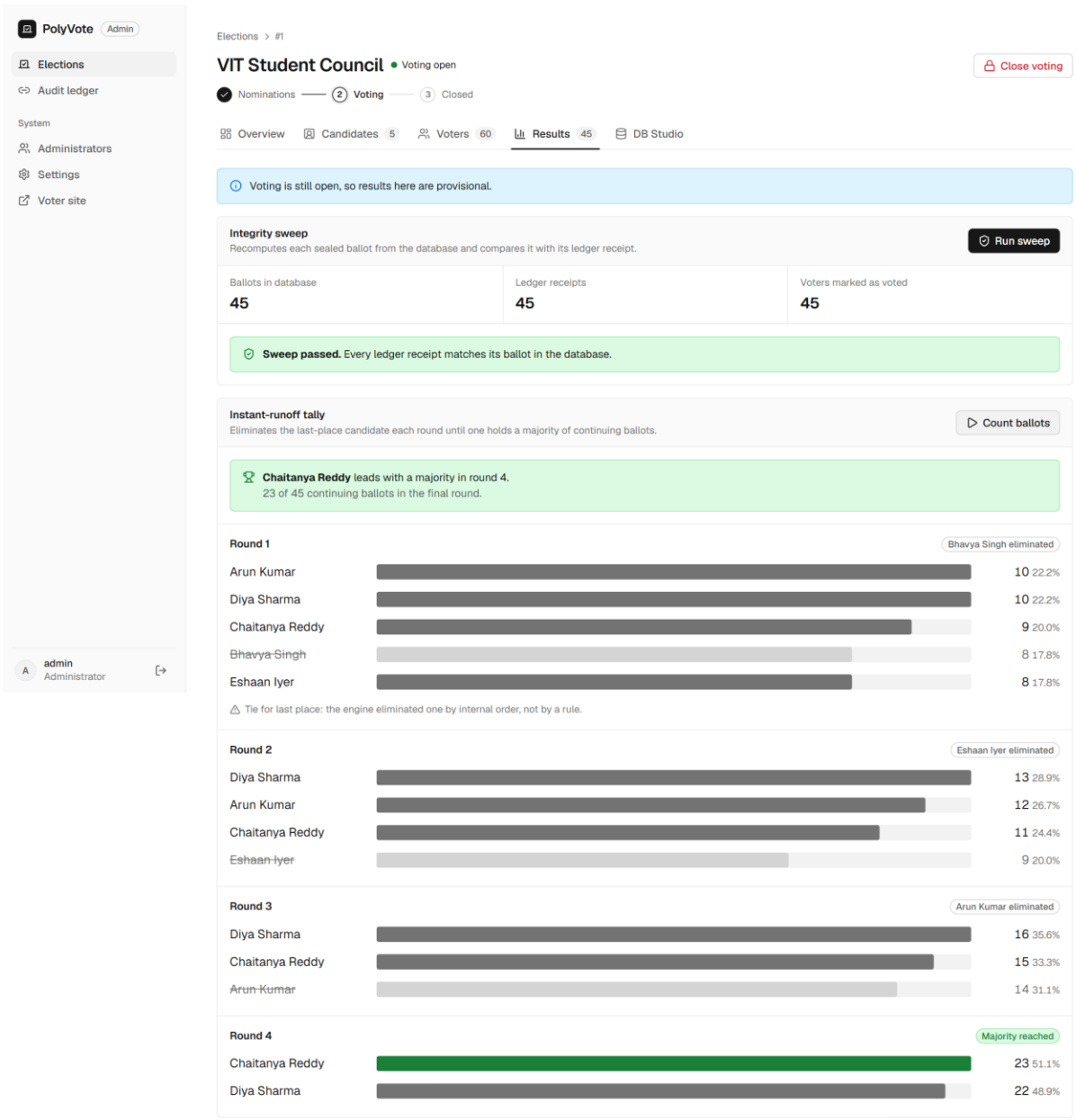


Figure 13: Integrity sweep passed and instant-runoff rounds (before tampering)

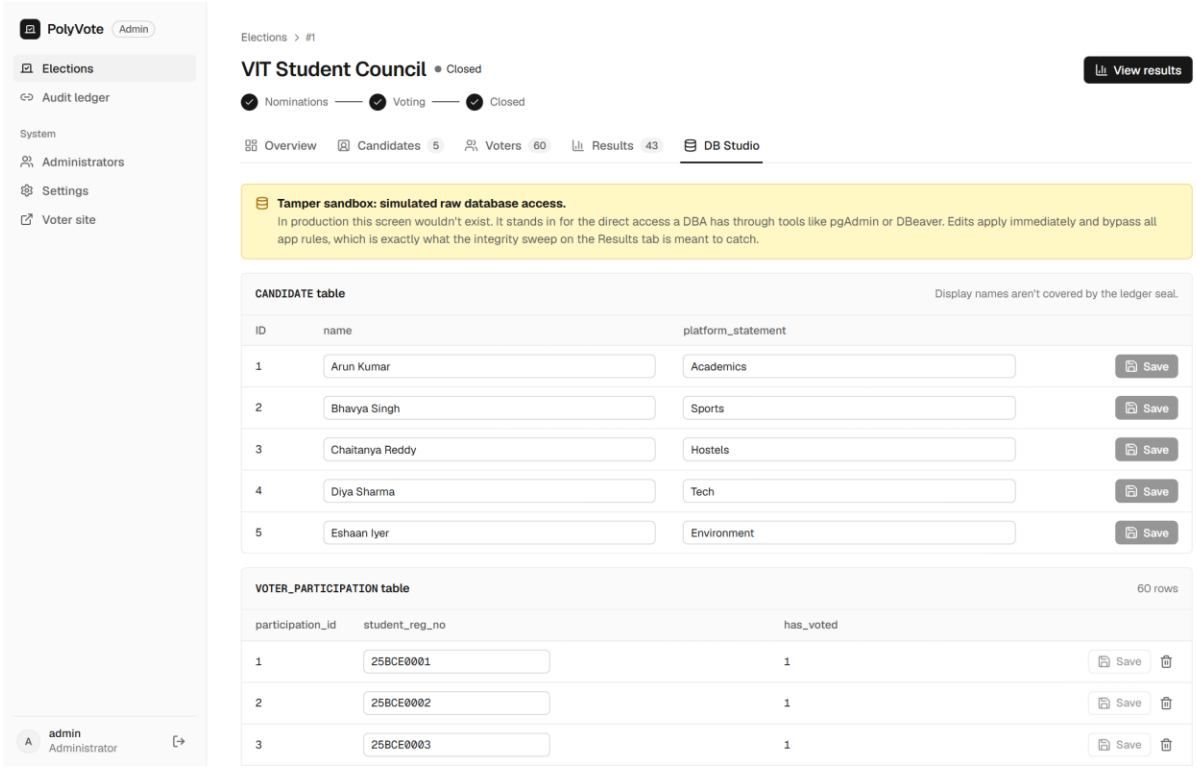


Figure 14: DB Studio: simulated rogue DBA editing and deleting ballot rows

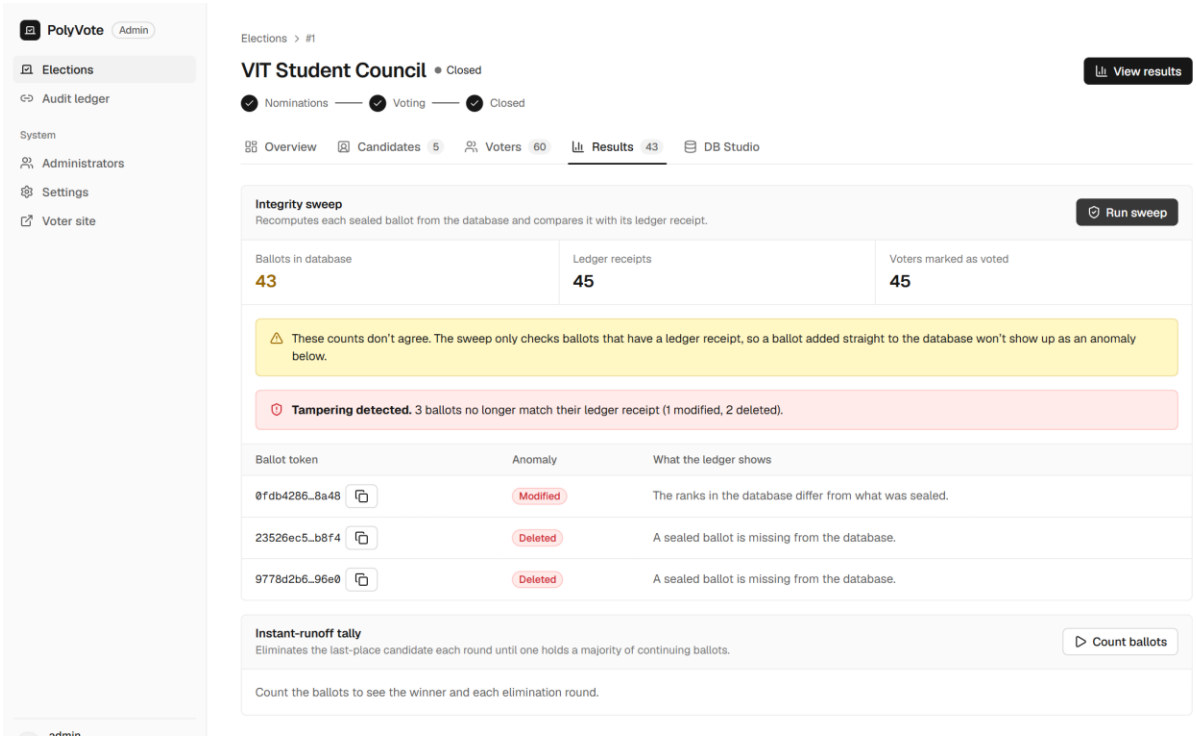


Figure 15: Integrity sweep after tampering: 1 modified and 2 deleted ballots detected

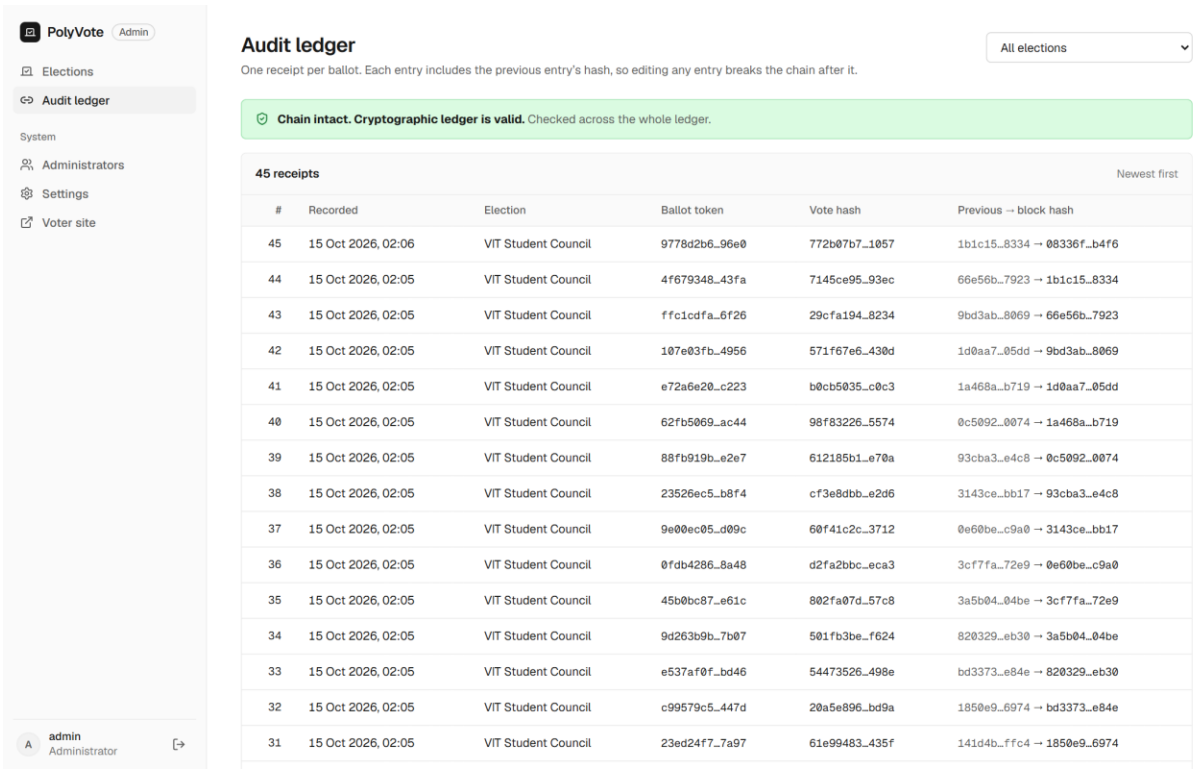


Figure 16: Audit ledger explorer: chain intact, receipts linked by hash

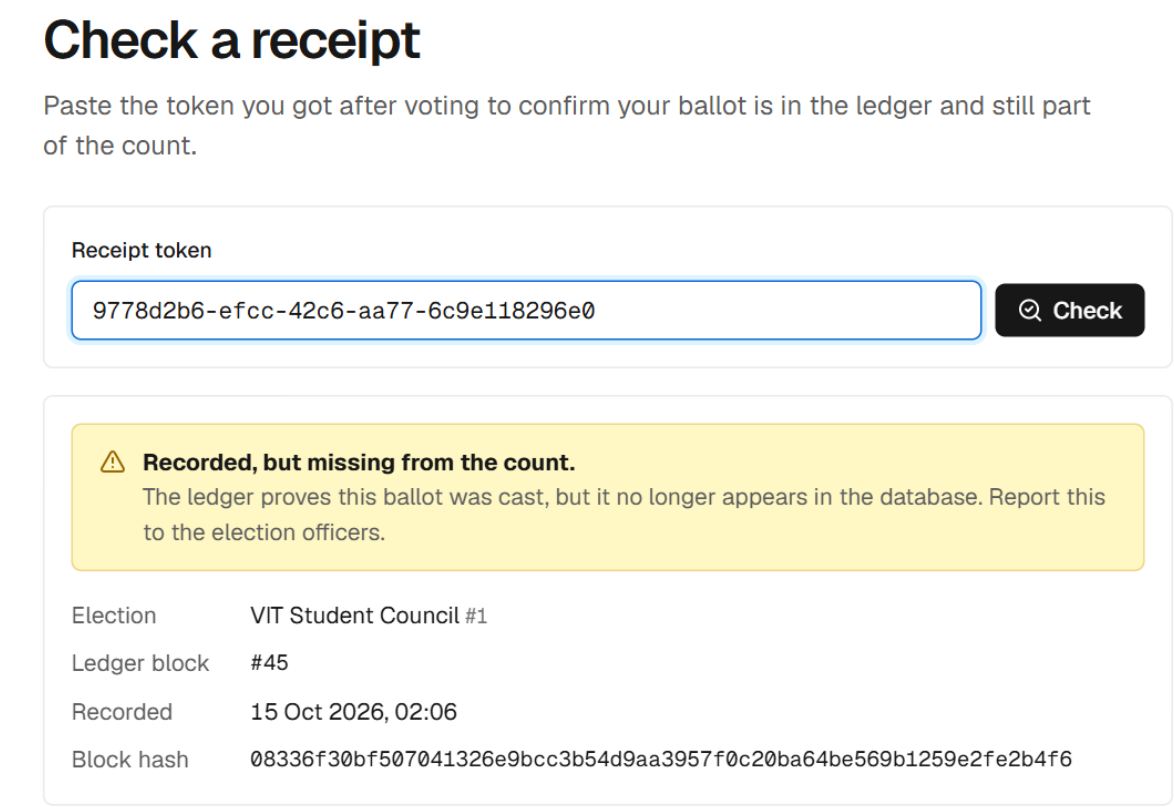


Figure 17: Voter receipt check after deletion: recorded but missing from the count

13. Changes Since Review I and Remaining Work

Area	Review I plan	Review II implementation	Reason
Relational DBMS	Oracle Autonomous DB, PL/SQL	SQLite with triggers, views and constraints	Zero-setup local development; schema kept portable for the Oracle migration.
NoSQL ledger	MongoDB Atlas	Hash-chained JSON documents (MongoDB document model)	Same document structure without a hosted cluster during development.
Frontend	Streamlit	React + TypeScript with a FastAPI REST layer; Streamlit retained	Separate voter and admin experiences, mobile support, clearer audit views.
Schema	Five entities (+POSITION listed)	Five entities + ADMIN_USERS; POSITION deferred	Officer logins needed a table; one post per election is enough for the demo.

Area	Review I plan	Review II implementation	Reason
Election lifecycle	Timestamp-driven stored procedures	Guarded transitions NOMINATION → VOTING → CLOSED	SQLite has no stored procedures; procedures follow with Oracle.
Audit	Hash-chain verification script	Integrity sweep, chain check, count reconciliation, DB Studio, voter receipt check	Makes the tamper demonstration visible to voters and evaluators.

Remaining work for the final review:

- Migrate the relational layer to Oracle Autonomous Database with PL/SQL procedures for automatic opening and closing, and DCL roles (GRANT/REVOKE) for officers and auditors.
- Move the ledger collection to MongoDB Atlas (the append and verification code already uses the document format).
- Deploy the application as a live, publicly accessible service.
- Add the POSITION entity for multi-post elections, a documented IRV tie-break rule, and a check that flags ballots present in the database without a ledger receipt.